

V8 INTERNALS / TORQUE DSL

V8のTorqueを 語りたい

西 悠太 / 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

株式会社ダイニー / Platform Engineer

TypeScriptが好きです。
V8の中身をのぞくのも好きです。

@riya-amemiya



「V8はC++で書かれている」

確かにほとんどの実装はC++です。

builtinsをのぞいてみると、そこにあるのは
C++でもJavaScriptでもない、見慣れない言語。

Torque

これがTorqueです

```
// src/builtins/array-foreach.tq より抜粋
transitioning javascript builtin ArrayForEach(
  js-implicit context: NativeContext, receiver: JSAny)(
    ...arguments): JSAny {
  const o: JSReceiver = ToObject_Inline(context, receiver);
  const len: Number = GetLengthProperty(o);
  const callbackfn = Cast<Callable>(arguments[0])
    otherwise TypeError;
  // ...
}
```

なぜGoogleは、
V8専用の言語を
わざわざ作ったのか？

01

Torque以前
CSAの時代

builtinsは手書きされていた

かつてV8のbuiltinsは、[CodeStubAssembler](#) (CSA) というC++ベースの低水準な記法で手書きされていました。

CSAは速い。けれど...

CSAの強み

- 関数呼び出しの余計なコストを避けられる
- 機械語レベルで攻めた最適化ができる
- 1つの記述で全アーキ向けに生成できる

CSAの弱み

- 型の保証が弱い
- 制御フローが手続き的で読みにくい
- 正しく書くのに深い専門知識が要る

一番の問題は、型の保証が弱いこと

```
TNode<Object> elem = LoadFixedArrayElement(elements, index);
```

```
TNode<JSArray> arr = UncheckedCast<JSArray>(elem);
```

CSAにも `TNode<T>` という型はあります。

ただ無検査のキャストが通るため、`elem` が本当に `JSArray` かまでは保証されません。

取り違えは、脆弱性になる

型の取り違えが容易に起こり、しかもそれがクラッシュや深刻な脆弱性の原因になっていました。

型ごとにメモリ上の配置が違うため、誤った型として読むと、無関係な領域をフィールドとして読み書きしてしまうためです。

つまり、こういう状況

安全で速いコードをCSAで書くには、

多くの専門知識が求められていました。

02

Torqueの登場
型のある世界へ

Torque = 静的型付きのDSL

CSA

- × 型の保証が弱い
- × 無検査キャストで取り違えが起きうる
- × 専門知識が前提

Torque

- V8の値に静的な型が付く
- 取り違えをコンパイル時に検出しやすい
- 仕様に近い書き方ができる

仕様の手順と対応づけて書ける

ECMAScript 仕様 (forEach)

1. Let `0` be ? ToObject(this value).
2. Let `len` be ? LengthOfArrayLike(O).
3. If IsCallable(callbackfn) is false, throw a `TypeError`.

Torque

```
const o: JSReceiver =  
    ToObject_Inline(context, receiver);  
const len: Number =  
    GetLengthProperty(o);  
const callbackfn =  
    Cast<Callable>(arguments[0])  
    otherwise TypeError;
```

失敗経路を、コンパイル時に強制

```
const arr = Cast<JSArray>(obj) otherwise Bailout;
```

```
typeswitch (n) {  
  case (s: Smi): { ... }  
  case (h: HeapNumber): { ... }  
}
```

`Cast` は実行時に型を検査し、失敗経路を書かないとコンパイルが通りません。

`typeswitch` は共用型を型ごとに分け、網羅性まで検査されます。

03

でも、
速さはどうなる？

Torqueはコンパイルされる



Torqueは実行時に読まれる言語ではありません。

コンパイルでCSAを呼ぶC++になり、そこから機械語まで落ちます。

しかも、ビルド時に機械語化

builtins の機械語は、V8 をビルドする時に
`mksnapshot` という工程で生成されます。

起動後は生成済みのコードを使うだけ。
Torqueを実行時に解釈するコストはありません。

だから、速さを犠牲にしない

Torqueは読みやすさと型安全を足しながら、
CSAの性能を引き継ぐ

04

まとめ

Torqueが同時に狙ったもの

読みやすさ

仕様の手順と対応づけて書ける

安全性

型の取り違えをコンパイル時に検出しやすい

速度

CSAの性能を引き継ぐ

この3つを同時に満たすために、Googleは専用言語という重い選択をしました。

今日覚えて帰ってほしいこと

- ✓ V8 の builtins の多くは Torque という専用言語で書かれている
- ✓ CSA の手書きは速いが、型の保証が弱かった
- ✓ Torque は静的型付きで、取り違えをコンパイル時に検出しやすい
- ✓ コンパイルで CSA を使うコードになるため、速さを犠牲にしない

參考資料

- V8 Docs: [Torque user manual](#)
- V8 Docs: [Torque builtins](#)
- V8 Blog: [Taming architecture complexity in V8 — the CodeStubAssembler](#)

ご清聴ありがとうございました

「V8はC++で書かれている」。

その内側には、Torqueがいる。